## ① Prac Final

- all students w/ GPA ≥ 3.5

```
SELECT * FROM students
WHERE gpa >= 3.5;
```

$\sigma_{gpa >= 3.5}(students)$

- customers(id, name)
  orders(id, customer_id, amount)
  → join customers w/ their orders (only customers who have orders)

a)
```
SELECT * FROM customers AS c, orders AS o
WHERE c.id = o.customer_id;
```

b)
```
SELECT * FROM customers AS c
INNER JOIN orders AS o
ON c.id = o.customer_id;
```
} 2 join options

* NOT LEFT JOIN bc we don't want all customers

$customers \bowtie_{customers.id = orders.customer\_id} (orders)$

- students(id, name, email) & email has NULLS.
  → count how many students have non-NULL email

```
SELECT COUNT(email) FROM students;
```
exclude null

~~COUNT(*)~~ include nulls + duplicates

COUNT(DISTINCT email) unique only

### True/False

- Either network under (H) heavy load or (F) system failure
  If system failure, notify incident response (N)
  → Engineer confirm system is NOT under heavy load. Then system must have notified incident response.
  True  Not H means F. F→N

- Given: $(P \to Q) \wedge (R \to \neg Q)$ then $P \to \neg R$  True

| P | Q | R | ¬Q | ¬R | P→Q | ∧ | R→¬Q | P→¬R | ∧→(P→¬R) |
|---|---|---|---|---|---|---|---|---|---|
| T | T | T | F | F | T | F | F | F | T |
| T | F | T | T | F | F | F | T | F | T |
| T | T | F | F | T | T | T | T | T | T |
| T | F | T | T | F | F | F | T | T | T |
| F | T | T | F | F | T | F | F | T | T |
| F | F | T | T | F | T | T | T | T | T |
| F | T | F | F | T | T | T | T | T | T |
| F | F | F | T | T | T | T | T | T | T |

$R \to \neg Q \leftrightarrow Q \to \neg R$

$(P \to Q) \wedge (Q \to \neg R) \Rightarrow P \to \neg R$

- Relational Algebra
  1. Student(SID, Lname, Fname, MI, GPA, Major)
  2. Course(CID, Cname, Dept)
  3. Grade(SID*, CID*, semester, Grade)

Find all course names + semesters where Oski Bear got grade 'A'

$OB\_SID \leftarrow \pi_{SID}(\sigma_{Lname='Bear' \text{ AND } Fname='Oski'}(Student))$

$A\_GRADE \leftarrow \sigma_{grade='A'}(Grade)$

$OB\_TAKEN \leftarrow OB\_SID \bowtie A\_GRADE$

$R \leftarrow \pi_{cname, semester}(OB\_TAKEN \bowtie Course)$

## ⑤ Relational Alg

- Projection (π) = pick cols
  $\pi_{id, citation}(stops)$
- Selection (σ) = filter rows by cond.
  $\pi_{id, citation}(\sigma_{age \le 50}(stops))$
- renaming (ρ)
  $\rho_{stops\_data(id \to person\_id)}(stops)$
- cartesian product (×)
  ↳ merge col from 2 rel + pairwise
  $\sigma_{stops.location = zips.location}(stops \times zips)$
- Theta Join ($\bowtie_\theta$)
  ↳ cartesian w/ select cond.
  $a \bowtie_{a.col2 > b.col2} B$
- Equi Join
  ↳ Theta w/ equality cond
  $stops \bowtie_{stops.location = zips.location} (Zips)$
- Natural Join
  ↳ 2 relations share attr name + type
  $stops \bowtie zips$
- Left/Right Outer Join
  $people \bowtie menu$ / $people \bowtie menu$
- grouping/agg
  ↳ $\gamma_{groupattrs;\ aggfunc(attr) \to newName(R)}$
  $\gamma_{major,\ count(SID) \to numstudents}(student)$

Find student SID who got 'A' in CID = 'CS101'

$R \leftarrow \pi_{SID}(\sigma_{grade='A' \text{ AND } CID='CS101'}(Grade))$

First, list name of everyone who took 'intro DB' in Fall 2024

$IDB \leftarrow \pi_{CID}(\sigma_{cname='intro\ DB'}(course))$

$FS \leftarrow \sigma_{semester='Fall\ 2024'}(Grade)$

$DB\_IN\_FALL \leftarrow IDB \bowtie_{CID=CID2} FS$

$R \leftarrow \pi_{Lname, Fname}(DB\_IN\_FALL \bowtie_{SID=SID} (Student))$

## ② SQL

- Add/drop column

```
ALTER TABLE [table_name]
DROP/ADD/MODIFY COLUMN [column_name] [datatype];
```
CHAR(15)

- Delete
  ↳ delete data from a table referenced by another table w/ a FK
  ON DELETE CASCADE
  - by condition
```
DELETE FROM [table_name]
WHERE [condition];
```
← ignore to delete ALL

- Update
```
UPDATE [table_name]
SET [condition]
WHERE [condition]
```

- UNION (ALL = duplicates)
  - combine 2+ select statements
  - of same # cols & data-types
- arithmetic on data in columns (Price * 0.1) As Discount)
- between
  - WHERE orderDate BETWEEN '1997-07-04' AND '1996-08-04'
  - inclusive range [a,b]
- ORDER BY [col_name] DESC/ASC
- Nested Queries

```
SELECT o.OrderID
FROM Orders AS o
WHERE o.CustomerID IN (
    SELECT c.CustomerID
    FROM Customer AS c
    WHERE c.CustomerName = 'Centro');
```

- Joins ← implicit inner/natural
  - 
```
SELECT FName, LName, o.OrderID, o.OrderDate
FROM Employee e, Orders o
WHERE e.EmployeeID = o.EmployeeID
```
(rows in both tables ↑)
  - 
```
SELECT [cols]
FROM [table t]
LEFT/RIGHT/INNER/OUTER JOIN [table t2]
    ON [conditions = conds]
```
keeps NULLS

↑ all rows on left kept

- Agg functions
  - count, avg, min, max, stddev, variance, sum
  - GROUP BY [col_names to be grouped by]
    HAVING [condition]
  - if col not in group by, add to SELECT alone
  - EX2:
```
SELECT p.prodName
FROM prod p, orderDetails od, Orders o
WHERE p.prodID = od.prodID AND od.orderID = o.orderID
GROUP BY p.prodName
HAVING COUNT(DISTINCT o.CID) >= 2
```

- DROP TABLE IF EXISTS [table_name];
- FK/PK
```
CREATE TABLE product(
    ProductID INTEGER PRIMARY KEY
        AUTO_INCREMENT,
    supplierID INTEGER
    FOREIGN KEY (supplierID) REFERENCES
        supplier(supplierID)
```
- Add row
```
INSERT INTO category VALUES (1, ...);
```
- Delete row
```
DELETE FROM Category
WHERE categoryID = 2
```
- UPDATE
```
UPDATE Category
SET Description = 'Tea, soda'
WHERE categoryID = 2
```
- Rename
```
ALTER TABLE Product
RENAME COLUMN Price TO unitprice
```
- SELECT-FROM-WHERE
- WHERE COND: =, <=, >=, !=, NULL, IS NOT NULL
  - WHERE country IN ('USA', 'CH')
    ~~OR~~ LIKE "T%"
  - NOT → AND → OR

## ③ Normalization

### 1NF

- R is in 1NF if all attributes are single-valued

R(SID, CID, semester, Ctitle, SLname, Major, Advisor, FacID, FaLname, classroom, OptionalTexts, Grade) } UNF

* OptionalTexts = sets ≠ atomic = violate 1NF

- Normalize into 2
  1) R1(SID, CID, semester, Ctitle, SLame, Major, Advisor, FacID, FacLname, Classroom, Grade)
  2) OptionalTextbooks (CID, Semester, Textbook)

### 2NF

R(A,B,C,D)
A→C ✗ AB→C ✓
B→C ✗

- relevant when R has composite key (CK = set of attributes to uniquely identify)
- all non-prime attributes must depend on the whole key only
- R is NOT in 2NF if some proper subset of a CK determines a non-prime attribute
  ↳ non-prime attributes = not in any CK
- R1(SID, CID, semester, Ctitle, SLname, Major, Advisor, FacID, FacLname, Classroom, Grade)

Problems

- insert anomalies = can't store FacID, FacLname, etc. w/o students signing up first
- deletion anomalies = lose faculty info if we del. student
- update anomalies = Ctitle, FacID, FacLname, Classroom repeats
  → if classroom changes, every record referring to CID/sem must be updated

- many non-key attributes don't depend on the whole key

| Non-prime attr. | Depends on | Why partial Dependent? |
|---|---|---|
| SLname | SID | partial of CK |
| Major | SID | partial of CK |
| Advisor | SID | partial of CK |
| Ctitle | CID | partial of CK |
| FacID | CID, semester | partial of CK |
| FacLname | CID, semester | partial of CK |
| Classroom | CID, semester | partial of CK |

⇒ only grade is not partially dependent

- Normalize R1 into 2NF
  1. Student_Lname (SID, SLname)
  2. Student Major (SID1, Major, Advisor)
  3. Course (CID, Ctitle)
  4. Cprof (CID3, semester, FacID, FacLname)
  5. CRoom (CID3, semester, classroom)
  6. Grade (SID1, CID3, Semester, Grade)

### 3NF

- a non-prime attribute can't depend on another non-prime attribute
- R is not in 3NF if ∃ nonprime pair A,B: A→B
- ⇔ R is 3NF if ∀ FD: X→Y satisfies:
  1) X is a candidate key OR
  2) Y is a prime attribute

4) CProf (CID3, semester, FacID, FacLname)

Problems

- can't store FacLname w/o course
- FacLname repeated every CID/semester
- if prof Lname changes, all will change
- ⇒ Transitive dependency
  {CID, semester} → FacID and FacID → FacLname
- ⇒ normalize
  4a) ProfName (FacID, FacLname)
  4b) CProfID (CID3, Semester, FacID

### BCNF

- if YX→Y, then X is a super-key [candidate key]
- BCNF → 3NF → 2NF → 1NF

## ④ FRQ

Enrollment_UNF:

| StudentID | StudentName | MajorID | MajorName | Courses |
|---|---|---|---|---|
| 1001 | Alice | M01 | CS | ((CS101, 'Intro DB', A), (CS102, 'Algo', B+)) |
| 1002 | Bob | M02 | EE | ((EE101, 'Circuits', B), (CS101, 'Intro DB', A-)) |
| 1003 | Carol | M01 | CS | ((CS101, 'Intro DB', B+), (MATH 201, 'Calc', A)) |

Assume FD

- StudentID → StudentName, MajorID
- MajorID → MajorName
- CourseID → CourseName
- StudentID, CourseID, → Grade (one grade per student-course pair

LHS = potential attributes to go in PK

1) Transform enrollment_UNF in 1NF. Show rows + underline PK

| StudentID | StudentName | MajorID | MajorName | CourseID | CourseName | Grade |
|---|---|---|---|---|---|---|
| 1001 | Alice | M01 | CS | CS101 | Intro DB | A |
| 1001 | Alice | M01 | CS | CS102 | Algo | B+ |
| 1002 | Bob | M02 | EE | EE101 | Circuits | B |
| 1002 | Bob | M02 | EE | CS101 | Intro DB | A- |
| 1003 | Carol | M01 | CS | CS101 | Intro DB | B+ |
| 1003 | Carol | M01 | CS | MATH201 | Calc | A |

2) From 1NF, produce 2NF + say which partial dependencies you removed

Student

| StudentID | StudentName | MajorID | MajorName |
|---|---|---|---|
| 1001 | Alice | M01 | CS |
| 1002 | Bob | M02 | EE |
| 1003 | Carol | M01 | CS |

Course

| CourseID | CourseName |
|---|---|
| CS101 | Intro DB |
| CS102 | Algo |
| EE101 | Circuits |
| MATH201 | Calc |

Enrollment

| StudentID | CourseID | Grade |
|---|---|---|
| 1001 | CS101 | A |
| 1001 | CS102 | B+ |
| 1002 | EE101 | B |
| 1002 | CS101 | A- |
| 1003 | CS101 | B+ |
| 1003 | MATH201 | A |

- removed partial dependencies:
  - StudentID → StudentName
  - StudentID → MajorID
  - CourseID → CourseName
  - MajorID → MajorName (transitive)
- Note: I kept (StudentID, CourseID → Grade) b/c not partial

3. Verify if 2NF tables are 3NF (by identifying at least one transitive dependency) If it isn't, decompose into 3NF relations, giving each relation + its PK. Show rows + PK

* NOT 3NF w/ transitive dep.

- StudentID → MajorID → MajorName (transitive)

T1

| StudentID | StudentName | MajorID |
|---|---|---|
| 1001 | Alice | M01 |
| 1002 | Bob | M02 |
| 1003 | Carol | M01 |

| MajorID | MajorName |
|---|---|
| M01 | CS |
| M02 | EE |

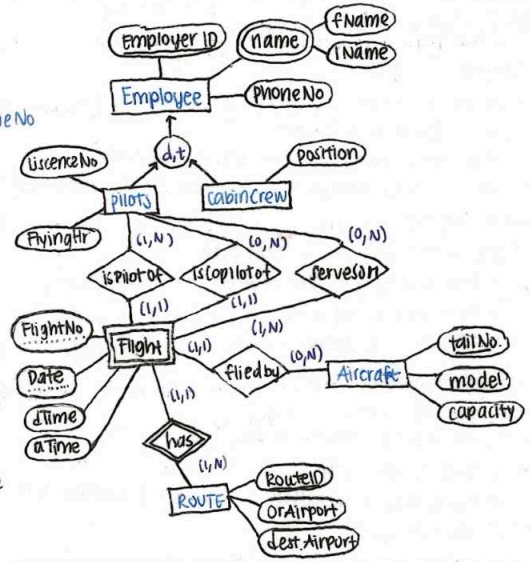
- course + enrollment tables are 3NF ⇒ depend on keys only

CS: Aanvi, Catherine, Jose, Rama, Richard, Sonia, Tam, Andre

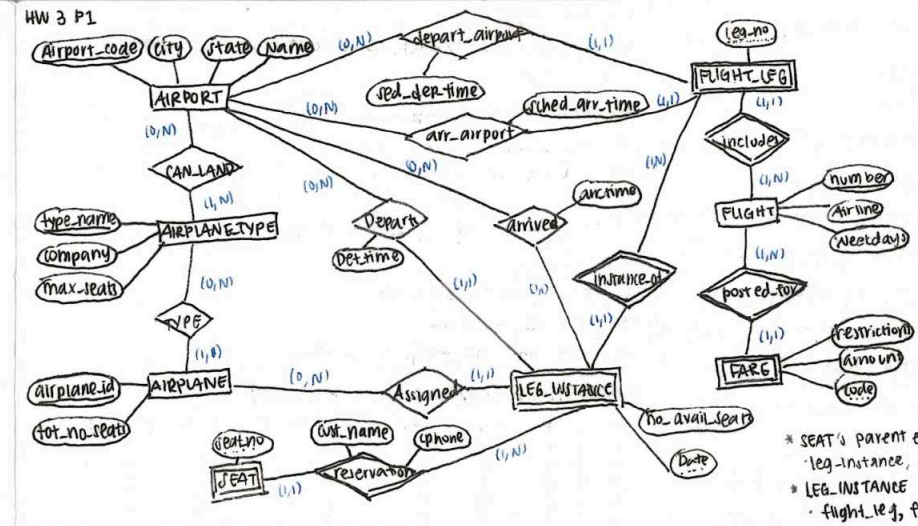
EE:

MT Q2 Airline DB

1. fleet of Aircraft, identified by unique tailNo. w/ model & capacity
2. employs diff types of Employees w/ unique employerID, name (composed of fistN, lastN), phoneNo
   - either Pilots/cabin crew → specialization
   - pilots store licenseNo, flyingHours
   - cabin crew stores position
3. operates routes w/ unique routeID & connects to specific originAirport & destinationAirport
4. operates flights, identified by flightNo that is unique only w/in its route, & date. Each flight has departureTime, arrTime
   - ea. flight assigned one aircraft, used for many flights or grounded
   - ea. flight assigned one pilot as captain & one as co-pilot. pilot can have many flights
   - ea. flight staffed by several cabin crew, which can be assigned to many flights, & at least one cabin crew assigned



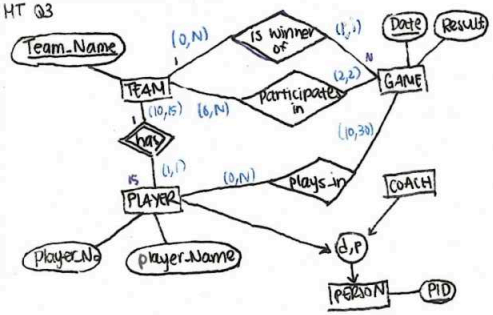

HW 3 P1




* SEAT's parent entity: leg_instance, flight_leg, flight
* LEG_INSTANCE
  - flight_leg, flight

1. AIRPORT (airport_code, city, state, name)
2. AIRPLANE_TYPE (airplane_type_name, company, max_seats)
3. AIRPLANE (airplane_id, tot_no_seats, airplane_type_name[2])
4. FLIGHT (flight_number, airline, weekdays)
5. FARE (code, flight_number[4], amount, restrictions)
6. FLIGHT_LEG (leg_no, flight_number[4], dep_airport_code[1], dep_time, arr_airport_code[1], arr_time)
7. LEG_INSTANCE (date, leg_number[6], flight_number[4], no_avail_seats, dep_airport_code[7], dep_time, arr_airport_code[7], arr_time, airplane_id[3])
8. SEAT (seat_no, date[7], leg_no[6], flight_number[4], cust_name, phone)
9. CAN_LAND (airplane_type_name[2], airport_code[1])

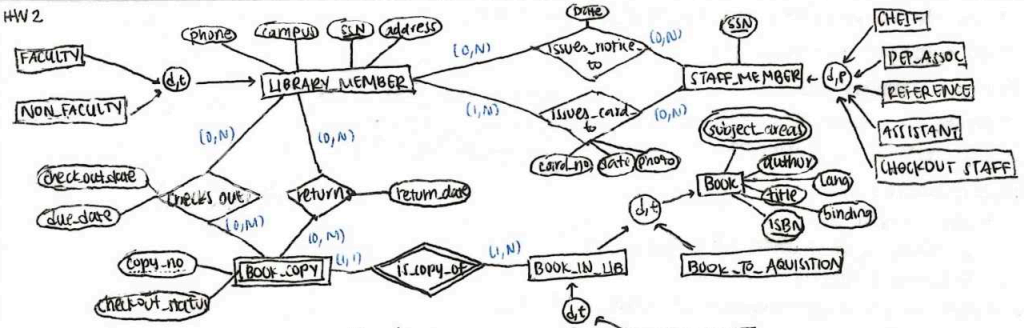
HT Q3




1. Person (PID)
   a. coach (PID[1])
   b. player (PID[1], player_No, player_Name, team_name[3])
2. Game (Date, result | win_team_Name[3])
3. Team (Team_Name)
4. PlaysIn (GameDate[2], PID[1b])
5. ParticipatesIn (GameDate[2], TeamName[3])

① strong entities → relation ② weak entities → relation + FK ③ 1:1 total → new, partial → add to total ④ 1:N add to N-side ⑤ M:N new relation, parents' PK : PK ⑥ multivalued new relation

HW 2




1. LIBRARY_MEMBER (SSN, phone, campus, address)
   1a. FACULTY (SSN[1])
   1b. NON_FACULTY (SSN[1])
2. STAFF_MEMBER (SSN)
   2a CHEIF (SSN[2])
   2b DEP_ASSOC (SSN[2])
   2c REFERENCE (SSN[2])
   2d ASSISTANT (SSN[2])
   2e CHECKOUT_STAFF (SSN[2])
3. BOOK (ISBN, title, author, lang, binding)
   3a. BOOK_TO_AQUISITION (ISBN[2])
   3b. BOOK_IN_LIB (ISBN[3])
      3bi. BOOK_CAN_BE_LOANED (ISBN[3])
      3bii. BOOK_CANNOT_BE_LOANED (ISBN[3])
4. BOOK_COPY (book_copy_no, ISBN[3b], checkout_status)
5. RETURNS (lib_mem_SSN[1], book_copy_no[4], book_ISBN[3b], return_date)
6. ISSUES_NOTICE_TO (staff_member_SSN[2], library_member_SSN[1], Date)
7. ISSUES_CARD_TO (staff_mem_SSN[2], lib_mem_SSN[1], card_no, date, photo)
8. SUBJECT_AREA (book_ISBN[3], subject_area_name)
9. CHECKS_OUT (lib_mem_SSN[1], book_copy_no[4], book_ISBN[3b], due_date)

## ① Inference Rules ⊨ true in all cases

- IR 1: $P \models P \vee Q$
- IR 2: $P \wedge Q \models P$
- IR 3: $P \wedge (P \to Q) \models Q$
- IR 4: $(P \to Q) \wedge (\neg Q) \models \neg P$
- IR 5: $(P \vee Q) \wedge (\neg P) \models Q$ ($P \vee Q$) $\wedge$ ($\neg Q$) $\overset{tp}{\models}$ Q
- IR 6: $(P \to Q) \wedge (Q \to R) \models P \to R$
- IR 7: $(P \to Q) \models (Q \to R) \to (P \to R)$
- IR 8: $(P \to Q) \wedge (R \to S) \models P \wedge R \to (Q \to S)$
- IR 9: $(P \vee Q) \wedge (\neg Q) \models P$ (IR 5)
- IR 10: $(P \to Q) \equiv \neg Q \to \neg P$
- IR 11: $(P \to Q) \equiv \neg P \vee Q$
- IR 12: $\neg (P \wedge Q) \equiv (\neg P \vee \neg Q)$
- IR 13: $\neg (P \vee Q) \equiv (\neg P \wedge \neg Q)$

HW2 Q1

$\neg (P \vee \neg Q) \vee (\neg P \wedge \neg Q) \equiv \neg P$

(1) assume $\neg P$

(2) $\neg (P \vee \neg Q) \equiv (\neg P \wedge Q)$ IR13

(3) $(\neg P \wedge Q) \vee (\neg P \wedge \neg Q)$

It is required that $\neg P$ & $\neg Q/Q$ doesn't matter

P Q ¬P ¬Q ... truth table

$P \vee Q \to R \equiv (P \to R) \wedge (Q \to R)$

true (from truth table)

$\{t \to p, \neg p \vee s\} \models \neg s \to \neg t$

(1) assume $\neg s$, then $\neg p$ must be true for $\neg p \vee s$ to be true

IR 9: $(\neg p \vee s) \wedge (\neg s) \models \neg p$

(2) if $\neg p$ true, then p is false and t is false for $t \to p$ to be true

IR 4: $(t \to p) \wedge (\neg p) \models \neg t$

(3) $\therefore \neg s \to \neg t$

Common Table

| P | Q | ¬P | ¬Q | P∧Q | P∨Q | P→Q |
|---|---|---|---|---|---|---|
| T | T | F | F | T | T | T |
| T | F | F | T | F | T | F |
| F | T | T | F | F | T | T |
| F | F | T | T | F | F | T |

- $(P \vee Q) \to R \equiv (P \to R) \wedge (Q \to R)$

- $(P \vee Q) \to (P \wedge Q)$ → False

| P | Q | P∨Q | P∧Q | (P∨Q)→(P∧Q) |
|---|---|---|---|---|
| T | T | T | T | T |
| T | F | T | F | F |
| F | T | T | F | F |
| F | F | F | F | T |

P=T, Q=F

P∨Q = T, but

P∧Q = F, so T→F false

PRAC MT Q1 (all True)

Given $(P \to Q)$ and $(\neg P \to Q)$, Q is tautological consequence

IR 8 , True

The premises $P \to Q$; $\neg R \to \neg Q$; $\neg R \vee S$ ; $\neg S$ entail $\neg P$

(1) assume $\neg S \to \neg P$, then $S \to P$

(2) if $P \to Q$, then Q

(3) if $\neg R \to \neg Q$, then R

(4) $\neg R \vee S \Leftrightarrow S$ since we have R $\therefore \neg S \to \neg P$

(1) If $\neg S \to \neg P$,

(2) $\neg R \vee S$ given $\neg S$ then $\neg R$

(3) $\neg R \to \neg Q$, then $\neg Q$

(4) $P \to Q$, then $\neg Q \to \neg P$ $\therefore \neg S \to \neg P$

$(P \to Q) \vee (P \to R)$ equivalent to $P \to (Q \vee R)$

| P | Q | R | P→Q | P→R | (P→Q)∨(P→R) | Q∨R | P→(Q∨R) |
|---|---|---|---|---|---|---|---|
| T | T | T | T | T | T | T | T |
| T | F | T | F | T | T | T | T |
| T | T | F | T | F | T | T | T |
| T | F | F | F | F | F | F | F |
| F | T | T | T | T | T | T | T |
| F | F | T | T | T | T | T | T |
| F | T | F | T | T | T | T | T |
| F | F | F | T | T | T | F | T |

(1) assume P, then Q

(2) assume P, then R

(3) Q ∨ R

(4) $P \to (Q \vee R)$ True

$(P \to (Q \to R))$ and $((P \wedge Q) \to R)$ equivalent

* note that F∧F is F

* truth table

$(P \vee Q) \to R \equiv \neg (P \vee Q) \vee R$ IR12

$\equiv (\neg P \wedge \neg Q) \vee R$ IR13

$\equiv (R \vee \neg P) \wedge (R \vee \neg Q)$ Distrib.

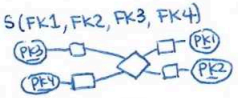
$\equiv (P \to R) \wedge (Q \to R)$ IR12

| P | Q | R | P∧Q | P∨Q | ¬R | P→R | (P∧Q)→R | Q→R |
|---|---|---|---|---|---|---|---|---|
| T | T | T | T | T | F | T | T | T |
| T | T | F | T | T | T | F | F | F |
| T | F | T | F | T | F | T | T | T |
| T | F | F | F | T | T | F | T | T |
| F | T | T | F | T | F | T | T | T |
| F | T | F | F | T | T | T | T | F |
| F | F | T | F | F | F | T | T | T |
| F | F | F | F | F | T | T | T | T |

- $P \to Q \equiv \neg P \vee Q$
- $\neg (P \wedge Q) \equiv \neg P \vee \neg Q$
- $\neg (P \vee Q) \equiv \neg P \wedge \neg Q$
- $(P \vee Q) \to R \equiv (P \to R) \wedge (Q \to R)$
- $P \to (Q \to R) \equiv (P \wedge Q) \to R$
- $(P \to Q) \wedge (\neg P \to Q) \equiv Q$
- $(P \vee Q) \to R \equiv (\neg P \wedge \neg Q) \vee R$

## ② EER → Relational Schema

- 1) Strong entities
  - Create relation
    - ↳ single-value attributes
    - ↳ primary key (PK) = entity's key underlined
    - ↳ composite attributes = split into simple attributes
- 2) weak entities
  - create relation like strong, plus...
    - ↳ add foreign key (FK) referencing parent entity
    - ↳ PK = partial key + FK
    - ↳ If owner is weak, include all PK attributes
- 3) 1:1 Relationships
  - total participation → min=1
  - partial participation → min=0
  - if both sides total:
    - ↳ merge/create relation for entities & relationship
  - if one side total:
    - ↳ add other side's PK as FK to total side
    - ↳ FK as PK or unique key
  - neither side total:
    - ↳ use multiple to multiple relationship
- 4) 1:N Relationships
  - add foreign key to N-side, referencing 1-side's PK
  - include relationship attributes in N-side
- 5) M:N Relationships
  - create new relation
  - both entities' PK as FK (to form new PK)
  - add relationship attributes
- 6) Multivalued Attributes (double circle)
  - create new relation
  - include owner's PK as FK + multivalued attr. itself
  - PK = (owner PK + attribute)
- 7) N-Ary Relationships
  - Create new table w/ all entities' PKs
  - add relationship attributes




- 8) Sub/superclass
  - Superclass w/ own PK
    - ↳ Subclass own relation w/ superclass PK
    - ↳ subclass specific attributes
  - works for all disjoint/overlapping, total/partial